

OBS1184

Advanced OpenTelemetry Collector Lab Guide

55-minute hands-on workshop | Splunk Distribution of OpenTelemetry Collector 0.157.0

Table of Contents

Prerequisites.....	2
Troubleshooting Connectivity.....	2
Access the Lab Environment.....	3
Description.....	3
Login to Splunk Show.....	3
Finding Your Instance Details:.....	5
Exercise 1 - Set Up and Validate the Agent.....	6
Description.....	6
Steps.....	6
Exercise 2 - Build the Configuration in Config Builder.....	6
Description.....	6
Steps.....	6
Exercise 3 - Deploy and Prove the Changes.....	7
Description.....	7
Steps.....	7

Prerequisites

Splunk's hands-on workshops are delivered via the [Splunk Show portal](#) - **you will need a splunk.com account in order to access this.**

If you don't already have a Splunk.com account, please create one [here](#) before proceeding with the rest of the workshop.

Troubleshooting Connectivity

If you experience connectivity issues with accessing either your workshop environment or the event page, please try the following troubleshooting steps. If you still experience issues please reach out to the team running your workshop.

- **Use Google Chrome** (if you're not already)
- If the event page (i.e. <https://splunk.show/...>) didn't load when you clicked on the link, try **refreshing the page**
- **Disconnect from VPN** (if you're using one)
- **Clear your browser cache and restart your browser** (if using Google Chrome, go to: Settings > Privacy and security > Delete browsing data)
- **Try using private/incognito browsing mode** to rule out any caching issues

Need help?

If you're still experiencing issues please ask a member of the onsite support team for assistance and they'll help get you up and running!

Access the Lab Environment

Description

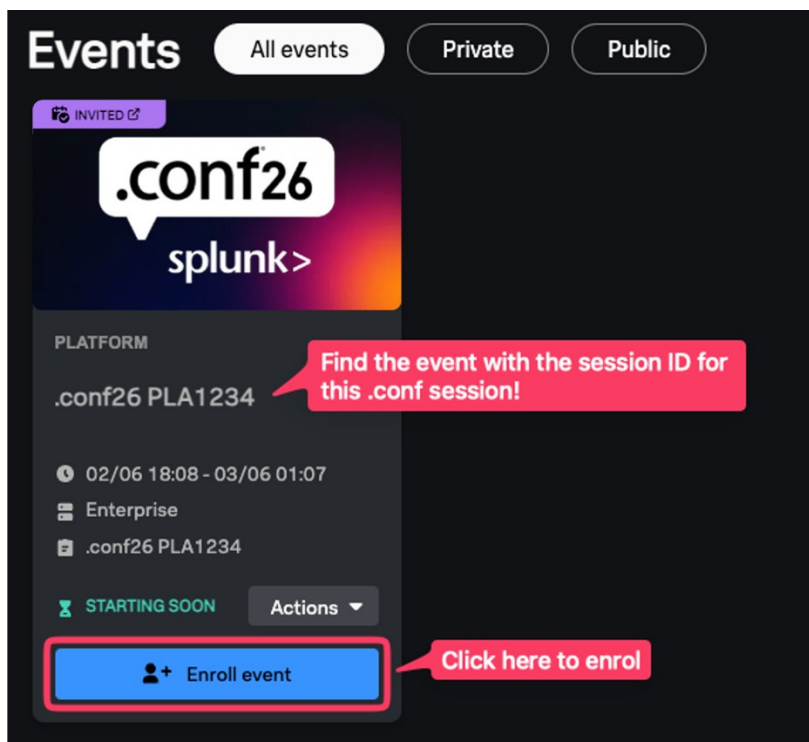
In this exercise, you will access your lab environment using our [Splunk Show portal](#).

Login to Splunk Show

You should have received an invite email from **Splunk Show** containing a link to a workshop event ("https://splunk.show/...")

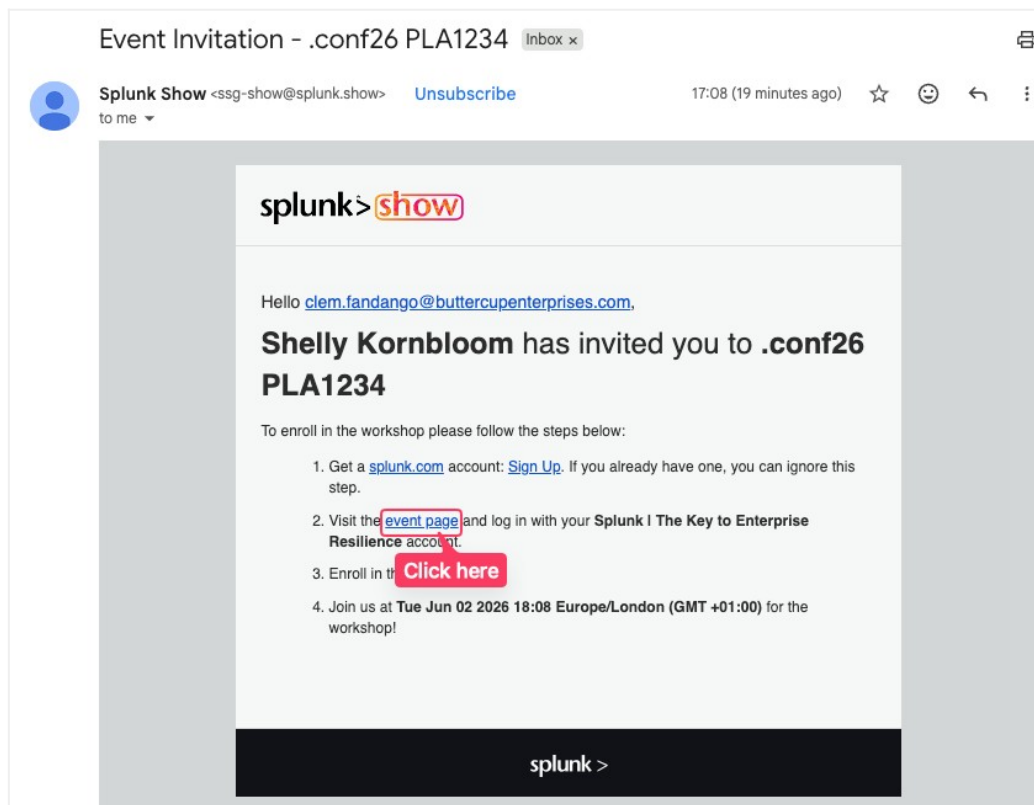
If you **DON'T** have an email from Splunk Show:

1. Please check your spam folder.
2. If you still can't find the email please visit <https://show.splunk.com> and log in with your splunk.com account.
3. Find the event for this session (look for the session ID, such as PLA1234, SEC5678, etc.) and click on **Enroll** event.

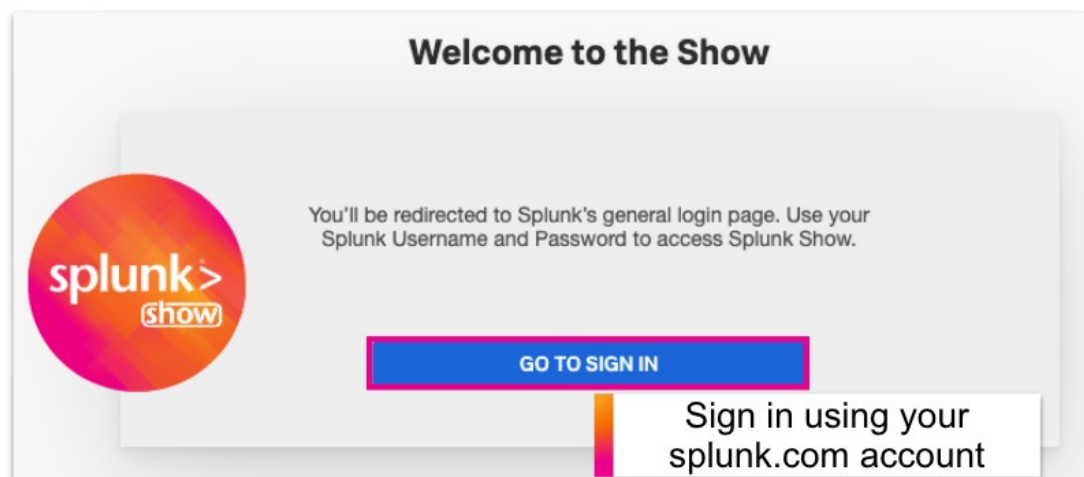


If you **DO** have an email from Splunk Show:

1. Open the email and click on the event link.



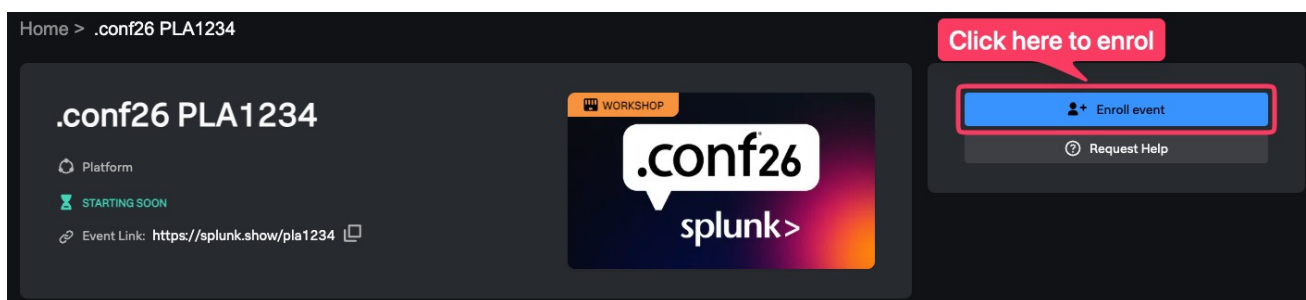
2. On the **Welcome to the Show** page click on **GO TO SIGN IN** and log in using your **Splunk.com account**.



Don't have a Splunk.com Account?

If you don't already have a Splunk.com account, don't worry - it only takes a few minutes to create one! Please create one [here](#).

3. Once logged in you should see the event page. Click on **Enroll event**.

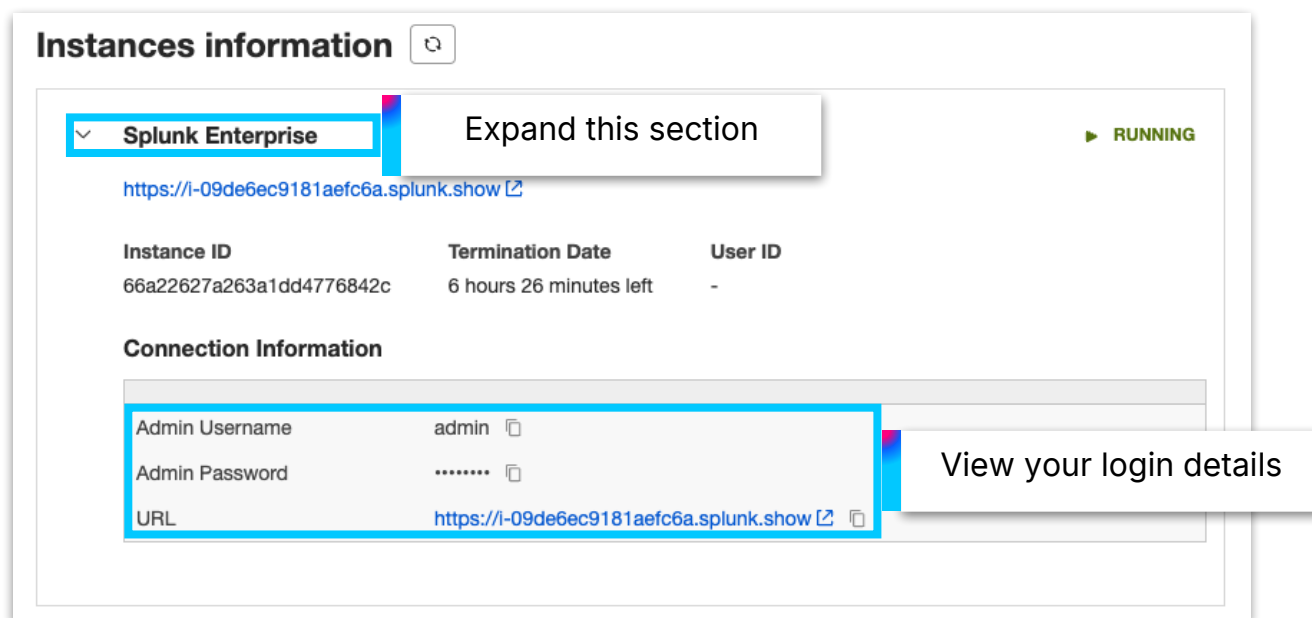


Need help enrolling?

Please ask a member of the onsite support team for assistance.

Finding Your Instance Details:

1. Scroll down the page to the **Instances information** section. Here you will find information on how to access your environment(s) along with the login details.



Note: The screenshot above shows 'Splunk Enterprise' but it may be called something else depending on the environments used in this hands-on session.

Exercise 1 - Set Up and Validate the Agent

Description

Prepare either a provided Linux workshop instance or an Apple Silicon Mac, run one Splunk OpenTelemetry Collector in Agent mode, generate baseline traces and host metrics, and import the starter configuration into OTel Collector Config Builder. The Collector is pinned to version 0.157.0.

Steps

1. Pair up if helpful, register for a free Splunk Observability Cloud organization, and open the online workshop page supplied for session OBS1184 in Google Chrome.

Registration: [Register for Splunk Observability Cloud Free](#)

Have your Observability Cloud **realm** and an **ingest access token** ready. The setup script stores them in `workshop-env.sh`; never share or commit that file.

2. Choose one supported execution path and confirm that `jq` is available with `jq --version`.
 - **Provided Linux instance:** use the Splunk Show instructions earlier in this guide, then connect with the SSH host, user, and port supplied for the session. Outbound access to port 2222 is required.

```
ssh -p 2222 <workshop-user>@<workshop-host>
```
 - **Apple Silicon Mac:** use Terminal locally. Apple Silicon supports the same local and cloud-connected exercises.
 - **Windows or Intel Mac:** use the provided Linux workshop instance; the local setup does not support these platforms.
3. Create the workshop directory.

```
mkdir advanced-otel-workshop && \  
+cd advanced-otel-workshop
```

This guide uses `[WORKSHOP]` for the full path to this directory.

4. Define the workshop download locations, then download the pinned Collector, the original workshop `loadgen`, and the setup script.

```
BRANCH=codex/advanced-collector-conf2026  
REPO=https://github.com/chentaow-splunk/observability-workshop/raw/refs/heads/$BRANCH  
LAB=content/en/ninja-workshops/foundations/2-opentelemetry-collector-workshops/2-advanced-  
collector-conf2026
```

Provided Linux instance

```
curl -L https://github.com/signalfx/splunk-otel-collector/releases/download/v0.157.0/  
otelcol_linux_amd64 -o otelcol  
curl -L "$REPO/workshop/ninja/advanced-otel/loadgen/build/loadgen-linux-amd64" -o loadgen  
curl -L "$REPO/$LAB/setup-workshop-conf2026.sh" -o setup-workshop.sh  
chmod +x setup-workshop.sh
```

Apple Silicon Mac

```
curl -L https://github.com/signalfx/splunk-otel-collector/releases/download/v0.157.0/otelcol_darwin_arm64 -o otelcol
curl -L "$REPO/workshop/ninja/advanced-otel/loadgen/build/loadgen-darwin-arm64" -o loadgen
curl -L "$REPO/$LAB/setup-workshop-conf2026.sh" -o setup-workshop.sh
chmod +x setup-workshop.sh
```

5. Run the setup script.

```
./setup-workshop.sh
```

Press Enter at the first prompt for the default cloud-connected mode. Enter your realm and ingest token, and accept the proposed API URL unless your organization specifies another endpoint. Press Enter to skip both optional HEC prompts; HEC is not required for the live workshop.

Choose local-only mode only when you intentionally do not want to send traces and metrics to Splunk Observability Cloud.

6. Verify that setup created the Agent files.

```
test -f 1-agent/agent_config.yaml && \
+test -f workshop-env.sh && \
+echo "Workshop setup is complete."
```

7. Open three terminals named **Agent Console**, **Loadgen**, and **Tests**. In Agent Console, start the Collector and leave it running.

```
cd [WORKSHOP]/1-agent
source ../workshop-env.sh
../otelcol --config=agent_config.yaml
```

Confirm that startup reaches Everything is ready. Begin running and processing data.

8. In Tests, verify the Agent health endpoint.

```
curl -fsS http://127.0.0.1:13133/
```

9. Verify baseline host metrics and traces.

After at least 10 seconds, the Agent Console should show `system.*` host metrics. Linux commonly includes CPU, memory, load, and network metrics; Apple Silicon can expose a different OS-supported subset.

```
cd [WORKSHOP]/1-agent
../loadgen -count 5
```

Confirm five `/movie-validator` spans in the Agent Console. The baseline span contains synthetic `user.*` attributes that you will protect later.

10. If cloud export is enabled, confirm the baseline in Splunk Observability Cloud.

Open **APM > Trace Analyzer**, select **All traces**, and find service `cinema-service` with operation `/movie-validator`. Then open **Infrastructure > Hosts** and locate the detected host. Cloud-provider and Kubernetes hosts can appear in their platform-specific navigator.

11. Import the starter `agent_config.yaml` into **Data Management > OTel Collector Config Builder**. Review Component Inventory, Pipelines, and Collector YAML.

The imported file combines Splunk Distribution defaults with workshop-only debug and file exporters. It uses one Agent and no Gateway. If the workshop runs remotely, download the same starter YAML from the online workshop page to the computer running your browser before importing it.

Note: Keep `workshop-env.sh` and all tokens outside Config Builder. Import only `agent_config.yaml`, which contains environment-variable references rather than credential values.

Exercise 2 - Build the Configuration in Config Builder

Description

Use the guided Config Builder interface to understand the imported Agent configuration and add three processing scenarios: drop noisy health spans, protect synthetic sensitive attributes, and transform structured logs. Keep all existing receivers, processors, exporters, extensions, and pipelines.

Steps

1. In **Component Inventory**, add processor filter with component name health, producing component ID filter/health.

Set top-level error_mode to ignore. Under trace_conditions, add one condition group and enter:

```
span.name == "/_healthz"
```

Leave the condition group's context and error mode empty. Preview the YAML and add the component. Use trace_conditions; the legacy traces.span form is deprecated.

2. Edit the traces pipeline and place filter/health immediately after memory_limiter. Keep every existing receiver, processor, and exporter.

```
processors:
  - memory_limiter
  - filter/health
  - resource/add_mode
  - batch
  - resource_detection
```

3. Add processor attributes. In Options, create these actions in order.

- Update user.phone_number to UNKNOWN NUMBER.
- Hash user.email.
- Delete user.password.

```
attributes:
  actions:
    - key: user.phone_number
      action: update
      value: UNKNOWN NUMBER
    - key: user.email
      action: hash
    - key: user.password
      action: delete
```

4. Add processor redaction. Set allow_all_keys to **True**, add the Visa and Mastercard patterns below to blocked_values, and set summary to debug.

```
\b4[0-9]{3}[\s-]?[0-9]{4}[\s-]?[0-9]{4}[\s-]?[0-9]{4}\b
\b5[1-5][0-9]{2}[\s-]?[0-9]{4}[\s-]?[0-9]{4}[\s-]?[0-9]{4}\b
```

The Amex value is intentionally unmatched so validation demonstrates an incomplete policy. Leave the database sanitizer and other optional fields unset.

5. Add attributes and redaction to the traces pipeline after filter/health and before

resource/add_mode.

```
traces:
  receivers: [jaeger, otlp, zipkin]
  processors: [memory_limiter, filter/health, attributes, redaction, resource/add_mode,
batch, resource_detection]
  exporters: [otlp_http, debug, file/traces]
```

6. Add processor transform. Set error_mode to ignore. Under log_statements, add a resource context containing:

```
keep_keys(resource.attributes, ["com.splunk.sourcetype", "host.name",
"otelcol.service.mode"])
```

Add a second log context with no statements, preview the component, and click **Add**. The resource statement removes metadata that is not required after collection.

7. Open **Collector YAML > Edit YAML**. Under the existing - context: log entry, paste this indented statements block and save only after the editor shows **VALID**.

```
statements:
  - set(log.cache, ParseJSON(log.body)) where IsMatch(log.body, "^\\{")
  - flatten(log.cache, "")
  - merge_maps(log.attributes, log.cache, "upsert")
  - set(log.severity_text, log.attributes["level"])
  - set(log.severity_number, 1) where log.severity_text == "TRACE"
  - set(log.severity_number, 5) where log.severity_text == "DEBUG"
  - set(log.severity_number, 9) where log.severity_text == "INFO"
  - set(log.severity_number, 13) where log.severity_text == "WARN"
  - set(log.severity_number, 17) where log.severity_text == "ERROR"
  - set(log.severity_number, 21) where log.severity_text == "FATAL"
```

ParseJSON creates a temporary map, flatten normalizes nested fields, and merge_maps(..., "upsert") promotes the parsed fields to log attributes. The remaining statements map application levels to OpenTelemetry severity.

8. Edit the logs pipeline and add transform after resource_detection. Keep all existing receivers and exporters.

```
logs:
  receivers: [fluent_forward, otlp, file_log/quotes]
  processors: [memory_limiter, resource/add_mode, batch, resource_detection, transform]
  exporters: [splunkhec, splunkhec/profiling, debug, file/logs]
```

9. Review the final generated YAML before download.

- filter/health, attributes, and redaction appear exactly once in traces.processors.
- transform appears exactly once in logs.processors, after resource_detection.
- The six imported pipelines and all existing components remain present.
- Cloud and local exporters remain connected to the traces and metrics pipelines.

Note: Processor order matters. Filtering first prevents dropped spans from reaching later processors. Resource detection must run before the log resource allowlist so host.name is available to keep. Promoting every JSON field can create high cardinality in production; use an explicit field allowlist for real workloads.

Exercise 3 - Deploy and Prove the Changes

Description

Download the completed YAML once, replace the running Agent configuration, and compare the original load-generator telemetry with the processed output. Validate traces and host metrics in Splunk Observability Cloud when cloud export is enabled. Logs and profiling remain take-home extensions.

Steps

1. Download the completed Collector YAML as `agent_config.yaml`.

Confirm that the file contains environment-variable references such as `${SPLUNK_ACCESS_TOKEN}`, not real credentials.

2. Stop the Agent with Ctrl-C, then preserve the starter configuration.

```
cd [WORKSHOP]/1-agent
if [ ! -f agent_config.start.yaml ]; then
  cp agent_config.yaml agent_config.start.yaml
fi
```

3. Replace the Agent configuration using the path that matches your environment.

Apple Silicon or same computer

```
cp ~/Downloads/agent_config.yaml [WORKSHOP]/1-agent/agent_config.yaml
```

Remote workshop instance - run from your local computer

```
scp -P 2222 ~/Downloads/agent_config.yaml \
+ <workshop-user>@<workshop-host>:~/advanced-otel-workshop/1-agent/agent_config.yaml
```

4. Move the earlier quote log and restart the updated Agent.

```
cd [WORKSHOP]/1-agent
if [ -f quotes.log ]; then
  mv quotes.log quotes.log.before-config-builder
fi
source ../workshop-env.sh
../otelcol --config=agent_config.yaml
```

Confirm that startup again reaches Everything is ready. Begin running and processing data.

5. In Loadgen, reuse the original workshop generator to create five application spans and five health-check spans. Copy one base trace ID.

```
cd [WORKSHOP]/1-agent
../loadgen -health -count 5
```

6. Compare the original and processed span counts locally.

- Original from loadgen: five `/movie-validator` and five `/_healthz` spans.
- Expected after the Collector: five `/movie-validator` spans and zero `/_healthz` spans.

```
jq -s -r '
[.[] .resourceSpans[].scopeSpans[].spans[]
| group_by(.name)[]
| "\(length) \(.[] .name)"
' agent-traces.out
```

Expected output: 5 `/movie-validator`.

7. Compare the original and protected span attributes.

- Phone: +1555-867-5309 → UNKNOWN NUMBER.
- Email: george@deathstar.email → deterministic SHA-256 hash.
- Password: LOTR>StarWars1-2-3 → deleted.
- Visa and Mastercard → ****; Amex remains visible by design.

```
jq -s '[
  .[].resourceSpans[].scopeSpans[].spans[].attributes[]
  | select(.key == "user.password")
] | length' agent-traces.out
```

Expected password count: 0.

8. When cloud export is enabled, prove the trace changes in Splunk APM.

Open **APM > Trace Analyzer**, select **All traces**, and choose a time range beginning just before the test. Search for the copied trace ID. The waterfall must contain /movie-validator and must not contain /_healthz. Select the application span and inspect its properties to confirm the protected values.

The positive application span proves this specific trace reached APM; the missing health span then proves filtering. If an organization hides a tag in the interface, use agent-traces.out as the authoritative attribute check.

9. Confirm that the unchanged host-metrics pipeline still reaches the backend.

```
jq -r '
  .resourceMetrics[].resource.attributes[]
  | select(.key == "host.name")
  | .value.stringValue
' agent-metrics.out | sort -u
```

Open **Infrastructure > Hosts**, find the detected host, and confirm that CPU, memory, load, or network data is current. Use the platform-specific Infrastructure navigator for EC2, Azure, GCP, or Kubernetes hosts.

Take-home: validate transformed logs locally

No HEC token is required. With the updated Agent running, generate five JSON quote logs and compare quotes.log with agent-logs.out.

```
../loadgen -logs -json -count 5
head -n 1 quotes.log
```

- level, message, movie, and timestamp become log attributes.
- DEBUG, INFO, WARN, and ERROR map to severity numbers 5, 9, 13, and 17.
- Only com.splunk.sourcetype, host.name, and otelcol.service.mode remain as resource attributes.

Take-home: extend collection

Logs to a non-production Splunk Platform instance: [Splunk HEC exporter documentation](#)

Correlate Platform logs with metrics and traces: [Introduction to Splunk Log Observer Connect](#)

Profiling: [Get data into Splunk APM AlwaysOn Profiling](#)

Note: A free Splunk Observability Cloud organization does not, by itself, provide a Splunk Platform HEC endpoint. Use a separate non-production Splunk Enterprise or Splunk Cloud Platform environment for the log-export take-home. Keep the final agent_config.yaml without secrets and review credentials, processor ordering, network exposure, and data volume before production use.